МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ імені ІГОРЯ СІКОРСЬКОГО»

Навчально-науковий фізико-технічний інститут

Кафедра інформаційної безпеки

ЗВІТ

з дисципліни «Кіберзахист об’єктів критичної інфраструктури»

Лабораторна робота № 5. Вдосконалення недоліків протоколів інтернету речей

Варіант 4 (парний): Modbus v3.1.6

Виконав: Вовкотруб Олександр Віталійович
Група: ФБ-61мн

Перевірив: Войцеховський А. В.

Київ 2026

Зміст

1 Мета роботи **2**

2 Постановка задачі **2**

3 Теоретичні відомості **2**

3.1 Протокол Modbus і його вразливості 2

3.2 Засоби усунення й обґрунтування вибору 3

4 Хід виконання **3**

4.1 Реалізація 3

4.2 Параметри захищеного з’єднання 4

5 Результати сніфінгу **4**

6 Активні атаки на канал зв’язку **5**

7 Ціна захисту **6**

8 Аналіз результатів **7**

9 Відповіді на контрольні питання **7**

10 Висновки **8**

11 Перелік джерел **8**

Додаток А. Клієнт/сервер Modbus TCP з TLS (`src/modbus_tcp.c`) **9**

Додаток Б. Проксі-сніфер (`src/sniffer.py`) **16**

Додаток В. Відтворення сценаріїв (`run.sh`) **19**

1 Мета роботи

Ознайомитися з недоліками протоколів пристроїв інтернету речей, що є частиною об’єктів критичної інфраструктури, і типовими програмними рішеннями з їх усунення. На прикладі протоколу Modbus впровадити криптографічний захист засобами бібліотеки OpenSSL і показати його дієвість за допомогою сніфера.

2 Постановка задачі

Згідно з варіантом (парний, Modbus v3.1.6):

1. проаналізувати вразливості протоколу Modbus;
2. запропонувати й обґрунтувати криптографічні засоби захисту;
3. реалізувати захищену взаємодію «клієнт-сервер» засобами OpenSSL;
4. показати дієвість захисту за допомогою сніфера, що перехоплює пакети, - порівняти трафік до та після впровадження захисту.

3 Теоретичні відомості

3.1 Протокол Modbus і його вразливості

Modbus, протокол прикладного рівня для промислової автоматизації (релейний захист, програмно-логічні контролери). Працює за моделлю «ведучий-ведений» (master-slave) з транзакціями «запит-відповідь». Основою є пакет PDU (Protocol Data Unit): однобайтовий код функції (1…127) і поле даних. Для передачі PDU загортається в ADU; у версії Modbus TCP перед PDU додається заголовок MBAP (ID транзакції, ID протоколу, довжина, адреса пристрою), а поле контрольної суми відсутнє (цілісність покладено на TCP).

Головні вразливості базового Modbus:

- **відсутність автентифікації**, сервер виконує будь-який коректно сформований запит, не перевіряючи, від кого він;
- **відсутність шифрування**, дані (коди функцій, адреси й значення регістрів) передаються у відкритому вигляді;
- **відсутність контролю цілісності** на рівні протоколу, немає підпису чи імітовставки, тому дані можна непомітно підмінити (атака «людина посередині», MITM).

Наслідок для об’єкта критичної інфраструктури: зловмисник, що має доступ до каналу зв’язку, може зчитувати технологічні параметри (розвідка) та підміняти команди й уставки контролерів, що безпосередньо загрожує технологічному процесу.

3.2 Засоби усунення й обґрунтування вибору

Оскільки сам Modbus не має механізмів безпеки, захист реалізують на транспортному рівні, загортаючи з'єднання в **TLS** (цей підхід стандартизовано як *Modbus/TCP Security* специфікацією Modbus Organization). TLS одразу закриває всі три вразливості:

- **конфіденційність**, симетричне шифрування корисного навантаження;
- **цілісність і автентичність повідомлень**, режим AEAD (вбудована імітовставка) виявляє будь-яку підміну;
- **автентифікація сторін**, за допомогою цифрових сертифікатів X.509, перевірених довіреним центром сертифікації (CA).

Вибір криптографічних примітивів (їх узгоджує TLS 1.3):

- **TLS 1.3** як мінімальна версія, прибрано застарілі й небезпечні набори шифрів, обов'язковий режим AEAD, швидше рукостискання (1-RTT);
- **ECDHE** (обмін ключами на еліптичних кривих з ефемерними ключами) - забезпечує *пряму секретність* (perfect forward secrecy): компрометація довгострокового ключа не розкриває раніше перехоплений трафік;
- **AES-256-GCM**, автентифікований симетричний шифр (AEAD): одночасно шифрування й контроль цілісності;
- **ECDSA на кривій P-256** для ключів сертифікатів, короткі ключі й швидке підписування, що важливо для обмежених ресурсів ПЛК;
- **автентифікація сервера за сертифікатом**, підписаним власним центром сертифікації об'єкта; клієнт відхиляє з'єднання, якщо сертифікат не підтверджено (захист від MITM).

4 Хід виконання

4.1 Реалізація

Реалізовано мінімальний Modbus TCP клієнт і сервер мовою C (`src/modbus_tcp.c`), що підтримують функцію 0x03 (Read Holding Registers) за специфікацією Modbus Application Protocol v1.1b3. Прапорець `--tls` загортає той самий прикладний обмін у TLS 1.3 засобами OpenSSL, не змінюючи логіку Modbus; завдяки цьому порівняння «до/після» коректне. Ввід-вивід абстраговано: функції `conn_read`/`conn_write` працюють або зі звичайним сокетом, або через `SSL_read`/`SSL_write`.

Сертифікати створено скриптом `gen-certs.sh`: власний кореневий CA (ECDSA P-256) і підписаний ним сертифікат сервера з ім'ям `plc.local` і SAN на 127.0.0.1. Клієнт перевіряє сертифікат сервера за `ca.crt` (`SSL_VERIFY_PEER`) і розриває з'єднання, якщо перевірка не пройшла.

Для перехоплення трафіку використано прозорий TCP-проксі-сніфер (`src/sniffer.py`), що моделює зловмисника на шляху «клієнт-сервер»: він пересилає байти між сторонами й

друкує їх у форматі hex. Такий сніфер бачить рівно те, що передається каналом (апаратний сніфер типу Wireshark у середовищі без прав на захоплення пакетів недоступний, тому застосовано проксі; результат для вмісту пакетів ідентичний). Обидва сценарії відтворюються скриптом `run.sh`.

4.2 Параметри захищеного з'єднання

Перевірка з'єднання (`openssl s_client`):

```
Protocol  : TLSv1.3
Cipher    : TLS_AES_256_GCM_SHA384
subject=  C=UA, O=Water Utility OKI, CN=plc.local
issuer=   C=UA, O=Water Utility OKI, CN=OKI Modbus CA
Verify return code: 0 (ok)
```

З'єднання використовує TLS 1.3 з AES-256-GCM, сертифікат сервера підтверджено ланцюжком до нашого CA.

5 Результати сніфінгу

Клієнт в обох сценаріях читає 4 регістри від адреси 0; сервер повертає значення 1000…1003. Перехоплений трафік наведено на рис. 5.1.

До захисту (відкритий Modbus TCP): сніфер бачить PDU і значення регістрів

```
[C->S] 12 байт
C->S 0000  00 01 00 00 00 06 01 03 00 00 00 04              ............
[S->C] 17 байт
S->C 0000  00 01 00 00 00 0b 01 03 08 03 e8 03 e9 03 ea 03  ................
S->C 0010  eb                                               .
```

Після захисту (Modbus через TLS 1.3): сніфер бачить лише шифротекст

```
[C->S] 1464 байт
C->S 0000  16 03 01 05 b3 01 00 05 af 03 03 46 99 72 83 cd  ...........F.r..
C->S 0010  7b 99 fb f7 1e 3e 9b bf b2 eb 58 77 ba 31 59 9a  {....>....Xw.1Y.
C->S 0020  98 fe 7f bc b5 16 3d 99 bb 84 bf 20 08 d9 4e 5a  ......=.... ..NZ
C->S 0030  f9 51 0d 48 86 5e be d9 1d 2c 09 2a 9e 17 0d 2c  .Q.H.^...,.*...,
C->S 0040  46 8a 98 5f 7a a4 bd a1 1d 06 c3 87 00 08 13 02  F.._z...........
C->S 0050  13 03 13 01 13 04 01 00 05 5e 00 0b 00 04 03 00  .........^......
C->S 0060  01 02 00 0a 00 1c 00 1a 11 ec 11 eb 11 ed 00 1d  ................
C->S 0070  00 17 00 1e 00 19 00 18 01 00 01 01 01 02 01 03  ................
C->S 0080  01 04 00 23 00 00 00 16 00 00 00 17 00 00 00 0d  ...#............
C->S 0090  00 24 00 22 09 04 09 05 09 06 04 03 05 03 06 03  .$."............
C->S 00a0  08 07 08 08 08 09 08 0a 08 0b 08 04 08 05 08 06  ................
C->S 00b0  04 01 05 01 06 01 00 2b 00 03 02 03 04 00 2d 00  .......+......-.
C->S 00c0  02 01 01 00 33 04 ea 04 e8 11 ec 04 c0 24 8c 24  ....3........$.
```

Рисунок 5.1. Трафік, перехоплений сніфером, до та після впровадження TLS

До захисту весь обмін читається у відкритому вигляді:

- запит `C->S: 00 01 00 00 00 06 01 03 00 00 00 04`, MBAP (ID транзакції `0001`, протокол `0000`, довжина `0006`, пристрій `01`) і PDU (код функції `03` = читання регістрів, початкова адреса `0000`, кількість `0004`);
- відповідь `S->C: ... 01 03 08 03 e8 03 e9 03 ea 03 eb`, код функції `03`, 8 байтів даних і значення регістрів `03 e8`…`03 eb`, тобто 1000…1003.

Зловмисник бачить і команду, і технологічні значення, і може їх підмінити.

Після захисту у трафіку видно лише рукостискання TLS і зашифровані записи. Автоматична перевірка перехоплених дампів підтверджує, що відкриті дані зникли (табл. 5.1).

Таблиця 5.1. Наявність відкритих даних Modbus у перехопленому трафіку

Шукана послідовність	До захисту	Після TLS
PDU запиту `01 03 00 00 00 04`	присутня	відсутня
Значення регістрів `03 e8 03 e9` (1000, 1001)	присутня	відсутня
Записи TLS (типи 16/14/17)	немає	ClientHello, ServerHello, ChangeCipherSpec, 11 записів Application Data (`17 03 03`)

Отже, після впровадження TLS сніфер не отримує жодної корисної інформації: ні команд, ні значень регістрів, ні можливості їх непомітно підмінити.

6 Активні атаки на канал зв’язку

Пасивне спостереження доводить лише втрату читабельності трафіку, тому сніфер доповнено трьома активними атаками (режими `--flip`, `--replay` та окремий підробний сервер). Режим `--flip` підробляє один байт у запиті клієнта (молодший байт початкової адреси регістрів 0 -> 8, тобто команда «читати регістри 0…3» перетворюється на «читати 8…11»); режим `--replay` після першої відповіді повторно надсилає серверу перехоплений запит.

Підміна команд. У відкритому Modbus атака вдається повністю: сервер виконує підроблену команду, і клієнт отримує значення регістрів 1008…1011 замість замовлених 1000…1003 (`data/client-plain-flip.txt`), тобто зловмисник непомітно керує тим, що читає (а в разі функції запису й тим, що виконує) контролер. У TLS той самий напад призводить до негайного розриву: сервер виявляє невідповідність імітовставки AEAD і надсилає попередження `bad record mac` (SSL alert 20), клієнт завершує роботу з помилкою (`data/client-tls-flip.txt`).

Повторне відтворення. Для відкритого Modbus повторений запит є законним запитом: сервер виконує його ще раз, і клієнт приймає дублікат відповіді (17 неочікуваних байтів з тими самими значеннями регістрів, `data/client-plain-replay.txt`). У TLS повторно надісланий запис Application Data відхиляється: номери записів уже витрачені, сервер розриває з'єднання, і клієнт не отримує нічого (`data/client-tls-replay.txt`).

Підробний сервер. Клієнт, що перевіряє сертифікат, відмовляється від з'єднання з сервером, сертифікат якого не підписано довіреним CA (`certificate verify failed`, `data/client-evil-server.txt`); без цієї перевірки саме так працювала би атака «людина посередині».

Таблиця 5.2. Результати активних атак до та після впровадження TLS

Атака	Відкритий Modbus	Modbus через TLS 1.3
Підміна байта команди	виконується непомітно (клієнт отримує інші регістри)	відхиляється, попередження `bad record mac`
Повторне відтворення запиту	виконується, клієнт отримує дублікат відповіді	відхиляється, сервер розриває з'єднання
Підробний сервер (невірний CA)	можливий (автентифікації немає)	клієнт відмовляється від з'єднання

7 Ціна захисту

Витрати на захист виміряно на 200 послідовних з'єднаниях по 20 запитів через проксі-сніфер на локальному вузлі (`--bench`, `data/bench.txt`).

Таблиця 5.3. Середній час встановлення з'єднання та запиту

Показник	Відкритий Modbus	Modbus через TLS 1.3
З'єднання з рукостисканням, мс	0,026	1,13
Запит-відповідь, мс	0,020	2,10

Рукостискання TLS 1.3 коштує близько 1 мс, а шифрування запису підіймає час обміну з ~0,02 до ~2 мс, тобто на два порядки в абсолютних величинах. Однак у реальному циклі опитування ПЛК (десятки-сотні мілісекунд) ці витрати непомітні, а рукостискання оплачується один раз на сесію; для обмежених контролерів частину навантаження можна перенести на прозорий шлюз. Отже, аргумент «TLS задорогий для ОТ» у цьому сценарії не підтверджується.

8 Аналіз результатів

Порівняння показує, що TLS усуває всі три вразливості базового Modbus (табл. 5.4), причому активні атаки (табл. 5.2) доводять це безпосередньо, а не лише наявністю шифротексту в дампі.

Таблиця 5.4. Властивості Modbus до та після захисту

Властивість	Базовий Modbus	Modbus через TLS 1.3
Конфіденційність	немає (відкритий текст)	AES-256-GCM
Цілісність повідомлень	немає (лише TCP)	AEAD (виявляє підміну)
Автентифікація сторін	немає	сертифікати X.509 + CA
Пряма секретність	немає	ECDHE

Ціна захисту виміряно безпосередньо (табл. 5.3): близько 1 мс на рукостискання і ~2 мс на запит, що непомітно в реальному циклі опитування. Для багатьох застарілих ПЛК, що не підтримують TLS, захист реалізують «прозорим шлюзом» (termination proxy) перед пристроєм. Варто зауважити обмеження: TLS із автентифікацією лише сервера не захищає від зловмисника, що видає себе за довірений клієнт, тому для команд запису специфікація Modbus/TCP Security передбачає взаємну автентифікацію (mTLS); повна модель загроз вимагає також сегментації мережі та контролю доступу (IEC 62443).

9 Відповіді на контрольні питання

1. Які вразливості має протокол Modbus. У базовому варіанті відсутні автентифікація сторін і шифрування; дані (коди функцій, адреси й значення регістрів) передаються у відкритому вигляді; немає контролю цілісності на рівні протоколу (у Modbus TCP немає навіть контрольної суми). Це уможливлює сніфінг, підміну команд і даних (MITM), несанкціоновані запити до контролерів, а також атаки повторного відтворення (replay). Додатково: функція читання/запису будь-яких регістрів без обмеження прав і відсутність захисту від перевантаження запитами (DoS).

2. Які засоби усунення вразливостей варто використовувати у більшості протоколів інтернету речей OKI. Основою є криптографічний захист каналу: шифрування корисного навантаження (симетричні AEAD-шифри, напр. AES-GCM), контроль цілісності (імітовставки HMAC або вбудовані в AEAD), автентифікація сторін за цифровими сертифікатами (X.509, PKI) або попередньо розподіленими ключами, обмін ключами з прямою секретністю (ECDHE), захист від повторного відтворення (лічильники, мітки часу, nonce). На практиці це реалізують стандартними протоколами, TLS/DTLS (для TCP/UDP), а для обмежених пристроїв відповідні профілі. Доповнюють їх організаційні й мережеві заходи: сегментація IT/OT, контроль доступу, оновлення прошивок, моніторинг трафіку (відповідно до IEC 62443).

10 Висновки

На прикладі протоколу Modbus впроваджено криптографічний захист засобами OpenSSL: той самий обмін Modbus TCP загорнуто в TLS 1.3 (AES-256-GCM, обмін ключами ECDHE, автентифікація сервера сертифікатом X.509, підписаним власним CA). Дієвість перевірено і пасивно, і активно: сніфер-зловмисник, що стоїть на шляху з'єднання, у відкритому режимі читав команди й значення регістрів у відкритому вигляді, а з активними атаками непомітно підміняв команду (клієнт отримав регістри 1008…1011 замість 1000…1003) і виконував повторне відтворення запитів (клієнт приймав дублікати відповідей). Після впровадження TLS усі три атаки провалилися: підміна виявляється імітовставкою AEAD (`bad record mac`), повторне відтворення відхиляється номерами записів, підробний сервер розпізнається перевіркою сертифіката, а в трафіку лишилися тільки рукостискання й зашифровані записи. Виміряна ціна захисту (~1 мс на рукостискання, ~2 мс на запит) непомітна в реальному циклі опитування ПЛК; для застарілих контролерів TLS впроваджують прозорим шлюзом, а для захисту від підробних клієнтів застосовують взаємну автентифікацію (mTLS, Modbus/TCP Security).

11 Перелік джерел

1. Modbus Organization. MODBUS Application Protocol Specification V1.1b3. 2012. URL: https://www.modbus.org/specs.php
2. Modbus Organization. MODBUS/TCP Security Protocol Specification. 2018.
3. libmodbus, A Modbus library (v3.1.6). URL: https://github.com/stephane/libmodbus
4. OpenSSL Project. OpenSSL 3.x Documentation. URL: https://www.openssl.org/docs/
5. Rescorla E. The Transport Layer Security (TLS) Protocol Version 1.3 : RFC 8446. IETF, 2018.
6. IEC 62443. Industrial communication networks. Network and system security.
7. Стьопочкіна І. В., Ільїн К. І., Овчарук М. В. Кіберзахист об'єктів критичної інфраструктури. Лабораторний практикум : навч. посіб. Київ : КПІ ім. Ігоря Сікорського, 2023. 59 с.

Додаток А. Клієнт/сервер Modbus TCP з TLS (`src/modbus_tcp.c`)

```
/*
 * Мінімальний Modbus TCP клієнтсервер/ із необовязковим' захистом TLS.
 *
 * Реалізує функцію 0x03 (Read Holding Registers) за специфікацією
 * Modbus Application Protocol v1.1b3 та Modbus Messaging on TCP/IP v1.0b.
 * Прапорець --tls загортає той самий обмін у TLS 1.3 (OpenSSL), не змінюючи
 * прикладну логіку: так видно різницю у трафіку до та після захисту.
 *
 *   server:  modbus_tcp --server [--tls] [--port N]
 *   client:  modbus_tcp --client [--tls] [--port N] [--addr 1 --count 4]
 */
#include <arpa/inet.h>
#include <netinet/in.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <unistd.h>

#include <openssl/err.h>
#include <openssl/ssl.h>

#define DEFAULT_PORT 1502 /* 502 потребує root; 1502 поширений для тестів */
#define NREG 16           /* кількість регістрів у сервера */

/* Один канал звязку': звичайний сокет або сокет, загорнутий у TLS. */
typedef struct {
    int fd;
    SSL *ssl; /* NULL, якщо TLS не використовується */
} conn_t;

static void die(const char *msg) {
    fprintf(stderr, "error: %s\n", msg);
    ERR_print_errors_fp(stderr);
    exit(1);
}

static int conn_read(conn_t *c, unsigned char *buf, int n) {
    return c->ssl ? SSL_read(c->ssl, buf, n) : (int)read(c->fd, buf, n);
}
static int conn_write(conn_t *c, const unsigned char *buf, int n) {
    return c->ssl ? SSL_write(c->ssl, buf, n) : (int)write(c->fd, buf, n);
}

/* Прочитати рівно n байтів (TCP може віддавати дані частинами). */
```

```c
static int read_full(conn_t *c, unsigned char *buf, int n) {
    int got = 0;
    while (got < n) {
        int r = conn_read(c, buf + got, n - got);
        if (r <= 0) return r;
        got += r;
    }
    return got;
}

/* ---- TLS ---- */
static SSL_CTX *tls_server_ctx(const char *cert, const char *key) {
    SSL_CTX *ctx = SSL_CTX_new(TLS_server_method());
    if (!ctx) die("SSL_CTX_new");
    SSL_CTX_set_min_proto_version(ctx, TLS1_3_VERSION);
    if (SSL_CTX_use_certificate_file(ctx, cert, SSL_FILETYPE_PEM) != 1)
        die("load cert запустіть( gen-certs.sh)");
    if (SSL_CTX_use_PrivateKey_file(ctx, key, SSL_FILETYPE_PEM) != 1)
        die("load key");
    return ctx;
}
static SSL_CTX *tls_client_ctx(void) {
    SSL_CTX *ctx = SSL_CTX_new(TLS_client_method());
    if (!ctx) die("SSL_CTX_new");
    SSL_CTX_set_min_proto_version(ctx, TLS1_3_VERSION);
    /* Автентифікація сервера за сертифікатом нашого CA. */
    if (SSL_CTX_load_verify_locations(ctx, "certs/ca.crt", NULL) != 1)
        die("load CA");
    SSL_CTX_set_verify(ctx, SSL_VERIFY_PEER, NULL);
    return ctx;
}

/* ---- Modbus ---- */

/* Відповідь сервера на один запит. Повертає 0, якщо клієнт відєднався'. */
static int serve_one(conn_t *c, unsigned short regs[]) {
    unsigned char req[260];
    if (read_full(c, req, 8) <= 0) return 0; /* MBAP(7) + код функції(1) */
    unsigned short len = (req[4] << 8) | req[5];
    int rest = len - 2; /* уже прочитано unit id + код функції */
    if (rest > 0 && read_full(c, req + 8, rest) <= 0) return 0;

    unsigned char fc = req[7];
    unsigned char resp[260];
    memcpy(resp, req, 4); /* transaction id + protocol id */
    resp[6] = req[6];     /* unit id */

    if (fc == 0x03) { /* Read Holding Registers */
        unsigned short start = (req[8] << 8) | req[9];
```

```
        unsigned short cnt = (req[10] << 8) | req[11];
        if (start + cnt > NREG || cnt == 0 || cnt > 125) {
            resp[7] = 0x83; /* exception */
            resp[8] = 0x02; /* illegal data address */
            unsigned short pl = 3;
            resp[4] = pl >> 8; resp[5] = pl & 0xff;
            return conn_write(c, resp, 9) > 0;
        }
        resp[7] = 0x03;
        resp[8] = cnt * 2; /* byte count */
        for (int i = 0; i < cnt; i++) {
            resp[9 + i * 2] = regs[start + i] >> 8;
            resp[9 + i * 2 + 1] = regs[start + i] & 0xff;
        }
        unsigned short pl = 3 + cnt * 2; /* unit + fc + bytecount + data */
        resp[4] = pl >> 8; resp[5] = pl & 0xff;
        return conn_write(c, resp, 9 + cnt * 2) > 0;
    }
    resp[7] = fc | 0x80; /* illegal function */
    resp[8] = 0x01;
    resp[4] = 0; resp[5] = 3;
    return conn_write(c, resp, 9) > 0;
}

static int run_server(int port, int use_tls, const char *cert, const char *key) {
    SSL_CTX *ctx = use_tls ? tls_server_ctx(cert, key) : NULL;
    unsigned short regs[NREG];
    for (int i = 0; i < NREG; i++) regs[i] = 1000 + i; /* показові значення */

    int ls = socket(AF_INET, SOCK_STREAM, 0);
    int on = 1;
    setsockopt(ls, SOL_SOCKET, SO_REUSEADDR, &on, sizeof on);
    struct sockaddr_in a = {.sin_family = AF_INET, .sin_port = htons(port),
                            .sin_addr.s_addr = htonl(INADDR_LOOPBACK)};
    if (bind(ls, (struct sockaddr *)&a, sizeof a) != 0) die("bind");
    if (listen(ls, 4) != 0) die("listen");
    fprintf(stderr, "server: %s on 127.0.0.1:%d\n", use_tls ? "TLS" : "plain", port);

    int fd;
    while ((fd = accept(ls, NULL, NULL)) >= 0) {
        conn_t c = {.fd = fd, .ssl = NULL};
        if (use_tls) {
            c.ssl = SSL_new(ctx);
            SSL_set_fd(c.ssl, fd);
            if (SSL_accept(c.ssl) != 1) { SSL_free(c.ssl); close(fd); continue; }
        }
        while (serve_one(&c, regs)) {}
        if (c.ssl) { SSL_shutdown(c.ssl); SSL_free(c.ssl); }
        close(fd);
```

```
    }
    close(ls);
    if (ctx) SSL_CTX_free(ctx);
    return 0;
}

static double now_ms(void) {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return ts.tv_sec * 1e3 + ts.tv_nsec / 1e6;
}

static void hexdump_fd(FILE *f, const unsigned char *buf, int n) {
    for (int i = 0; i < n; i++) fprintf(f, "%02x ", buf[i]);
    fprintf(f, "\n");
}

/* Один обмін запитвідповідь-; повертає 1 успіх, 0 кінець, -1 помилка. */
static int exchange(conn_t *c, unsigned short start, unsigned short cnt, int print) {
    unsigned char req[12] = {0x00, 0x01, 0x00, 0x00, 0x00, 0x06, 0x01,
                             0x03, start >> 8, start & 0xff, cnt >> 8, cnt & 0xff};
    if (conn_write(c, req, sizeof req) <= 0) return -1;
    unsigned char resp[9 + 255];  /* MBAP+PDU header (9) + max byte count (255) */
    if (read_full(c, resp, 9) <= 0) return -1;
    unsigned char bytecount = resp[8];
    if (bytecount == 0 || read_full(c, resp + 9, bytecount) <= 0) return -1;
    if (print) {
        printf("client: прочитано %d регістрів від адреси %u:\n", cnt, start);
        for (int i = 0; i < cnt; i++) {
            unsigned short v = (resp[9 + i * 2] << 8) | resp[9 + i * 2 + 1];
            printf("  reg[%u] = %u\n", start + i, v);
        }
    }
    return 1;
}

static int run_client(int port, int use_tls, unsigned short start, unsigned short cnt,
                      int repeat, int hold_ms) {
    SSL_CTX *ctx = use_tls ? tls_client_ctx() : NULL;
    int fd = socket(AF_INET, SOCK_STREAM, 0);
    struct sockaddr_in a = {.sin_family = AF_INET, .sin_port = htons(port),
                            .sin_addr.s_addr = htonl(INADDR_LOOPBACK)};
    if (connect(fd, (struct sockaddr *)&a, sizeof a) != 0) die("connect");
    conn_t c = {.fd = fd, .ssl = NULL};
    if (use_tls) {
        c.ssl = SSL_new(ctx);
        SSL_set_fd(c.ssl, fd);
        if (SSL_connect(c.ssl) != 1) die("TLS handshake");
        if (SSL_get_verify_result(c.ssl) != X509_V_OK) die("cert not trusted");
```

```
        printf("client: TLS %s, сервер автентифіковано\n", SSL_get_version(c.ssl));
    }

    for (int i = 0; i < repeat; i++)
        if (exchange(&c, start, cnt, 1) < 0) die("exchange failed");

    /* Тримати зєднання' відкритим, щоб побачити дані атаки повторного
       відтворення сніфер( --replay) або нічого не отримати. */
    if (hold_ms > 0) {
        struct timeval tv = {.tv_sec = hold_ms / 1000, .tv_usec = (hold_ms % 1000) * 1000};
        setsockopt(fd, SOL_SOCKET, SO_RCVTIMEO, &tv, sizeof tv);
        unsigned char extra[512];
        int r = conn_read(&c, extra, sizeof extra);
        if (r > 0) {
            printf("client: НЕОЧІКУВАНІ дані ознака( replay, %d байт): ", r);
            hexdump_fd(stdout, extra, r);
        } else if (errno == EAGAIN || errno == EWOULDBLOCK) {
            printf("client: додаткових даних не надійшло\n");
        } else {
            printf("client: зєднання' розірвано сервером після повторного "
                   "відтворення запису (TLS відхилив повтор)\n");
        }
    }
    if (c.ssl) { SSL_shutdown(c.ssl); SSL_free(c.ssl); }
    close(fd);
    return 0;
}

/* Вимірювання: час встановлення зєднання' (TCP або TCP+TLS) і час
   кругового обходу запиту. */
static int run_bench(int port, int use_tls, int nconn, unsigned short cnt) {
    SSL_CTX *bench_ctx = use_tls ? tls_client_ctx() : NULL;
    double t_conn = 0, t_rtt = 0;
    int ok = 0, rtts = 0;
    for (int i = 0; i < nconn; i++) {
        int fd = socket(AF_INET, SOCK_STREAM, 0);
        struct sockaddr_in a = {.sin_family = AF_INET, .sin_port = htons(port),
                                .sin_addr.s_addr = htonl(INADDR_LOOPBACK)};
        double t0 = now_ms();
        if (connect(fd, (struct sockaddr *)&a, sizeof a) != 0) { close(fd); continue; }
        conn_t c = {.fd = fd, .ssl = NULL};
        if (use_tls) {
            c.ssl = SSL_new(bench_ctx);
            SSL_set_fd(c.ssl, fd);
            if (SSL_connect(c.ssl) != 1 ||
                SSL_get_verify_result(c.ssl) != X509_V_OK) { close(fd); continue; }
        }
        t_conn += now_ms() - t0;
        for (int k = 0; k < 20; k++) {
```

```
            double t1 = now_ms();
            if (exchange(&c, 0, cnt, 0) == 1) { t_rtt += now_ms() - t1; rtts++; }
        }
        if (c.ssl) { SSL_shutdown(c.ssl); SSL_free(c.ssl); }
        close(fd);
        ok++;
    }
    if (!ok) die("bench: жодного зєднання'");
    printf("bench: %d зєднань', %s\n", ok, use_tls ? "TLS 1.3" : "plain TCP");
    printf("  середній час зєднання' з( рукостисканням): %.3f мс\n", t_conn / ok);
    if (rtts) printf("  середній час запитвідповідь-: %.3f мс (%d запитів)\n", t_rtt / rtts,
    ↪ rtts);
    return 0;
}

/* Значення прапорця, що йде наступним аргументом; якщо його немає, не падаємо. */
static const char *arg_val(int argc, char **argv, int *i) {
    if (*i + 1 >= argc) {
        fprintf(stderr, "error: %s requires a value\n", argv[*i]);
        exit(2);
    }
    return argv[++(*i)];
}

int main(int argc, char **argv) {
    int server = 0, client = 0, tls = 0, bench = 0, port = DEFAULT_PORT;
    int repeat = 1, hold_ms = 0;
    unsigned short start = 0, cnt = 4;
    const char *cert = "certs/server.crt", *key = "certs/server.key";
    for (int i = 1; i < argc; i++) {
        if (!strcmp(argv[i], "--server")) server = 1;
        else if (!strcmp(argv[i], "--client")) client = 1;
        else if (!strcmp(argv[i], "--tls")) tls = 1;
        else if (!strcmp(argv[i], "--bench")) bench = atoi(arg_val(argc, argv, &i));
        else if (!strcmp(argv[i], "--port")) port = atoi(arg_val(argc, argv, &i));
        else if (!strcmp(argv[i], "--addr")) start = atoi(arg_val(argc, argv, &i));
        else if (!strcmp(argv[i], "--count")) cnt = atoi(arg_val(argc, argv, &i));
        else if (!strcmp(argv[i], "--repeat")) repeat = atoi(arg_val(argc, argv, &i));
        else if (!strcmp(argv[i], "--hold")) hold_ms = atoi(arg_val(argc, argv, &i));
        else if (!strcmp(argv[i], "--cert")) cert = arg_val(argc, argv, &i);
        else if (!strcmp(argv[i], "--key")) key = arg_val(argc, argv, &i);
    }
    if (tls) { SSL_library_init(); SSL_load_error_strings(); }
    if (server) return run_server(port, tls, cert, key);
    if (bench > 0) return run_bench(port, tls, bench, cnt);
    if (client) return run_client(port, tls, start, cnt, repeat, hold_ms);
    fprintf(stderr, "usage: %s --server|--client [--tls] [--port N] [--bench N] "
            "[--repeat N] [--hold MS] [--cert F] [--key F]\n", argv[0]);
    return 2;
```

```
}
```

Додаток Б. Проксі-сніфер (`src/sniffer.py`)

```
"""Прозорий TCPпроксісніфер--: моделює зловмисника на шляху клієнт«сервер-».Слухає

 на LISTEN_PORT, зєднується' з SERVER_PORT і пересилає байти в обидвабоки
, друкуючи їх у форматі hex+ASCII. Для відкритого Modbus видно PDUкод
( функції, значення регістрів); для TLS видно лише шифротекст.Режими

 активних атак доводять( дієвість або безсилля зловмисника):
  --flip     зіпсувати один байт у першому запиті клієнта підміна( команди);
  --replay   повторно надіслати серверу перехоплений запит після першої
             відповіді атака( повторного відтворення).

    python sniffer.py --listen 1600 --server 1502 --out data/sniff.txt [--flip|--replay]
"""
from __future__ import annotations

import argparse
import socket
import threading


def hexdump(data: bytes, prefix: str) -> str:
    lines = []
    for off in range(0, len(data), 16):
        chunk = data[off:off + 16]
        hexs = " ".join(f"{b:02x}" for b in chunk)
        text = "".join(chr(b) if 32 <= b < 127 else "." for b in chunk)
        lines.append(f"{prefix} {off:04x}  {hexs:<47}  {text}")
    return "\n".join(lines)


def pump(src: socket.socket, dst: socket.socket, label: str, out) -> None:
    while True:
        try:
            data = src.recv(4096)
        except OSError:
            break
        if not data:
            break
        out.write(f"[{label}] {len(data)} байт\n{hexdump(data, label)}\n")
        out.flush()
        try:
            dst.sendall(data)
        except OSError:
            break
    dst.close()
```

```
def pump_attack(src: socket.socket, dst: socket.socket, label: str, out,
                mode: str, state: dict) -> None:
    """Пересилання з активною атакою: --flip підміна() або --replay."""
    peer = {"C->S": "клієнта", "S->C": "сервера"}[label]
    while True:
        try:
            data = src.recv(4096)
        except OSError:
            break
        if not data:
            break
        out.write(f"[{label}] {len(data)} байт\n{hexdump(data, label)}\n")
        out.flush()

        # перший чанк від клієнта визначає тип сесії: 0x16 = ClientHello (TLS)
        if label == "C->S" and "tls" not in state and data[:1] == b"\x16":
            state["tls"] = True
        is_plain_mb = (not state.get("tls") and len(data) >= 12
                       and data[7] == 0x03)  # відкритий Modbus PDU
        is_tls_rec = data[0] == 0x17                          # запис Application Data TLS
        if mode == "flip" and label == "C->S" and not state.get("flipped") \
                and (is_plain_mb or is_tls_rec):
            # для Modbus: байт 9, молодший біт початкової адреси регістрів
            # (0 -> 8): сервер виконає команду з ІНШОЮ адресою, ніж просив
            # клієнт; для TLS: зіпсований байт шифрозапису.
            tampered = bytearray(data)
            off = 9 if is_plain_mb else min(16, len(data) - 1)
            out.write(f"MITM: підроблено байт {off} даних {peer} "
                      f"({data[off]:02x} -> {tampered[off] ^ 0x08:02x})\n")
            out.flush()
            tampered[off] ^= 0x08
            data = bytes(tampered)
            state["flipped"] = True

        if mode == "replay" and label == "C->S" and not state.get("captured") \
                and (is_plain_mb or is_tls_rec):
            state["captured"] = bytes(data)
            state["srv_sock"] = dst  # серверний бік зєднання'

        if mode == "replay" and label == "S->C" and state.get("captured") \
                and not state.get("replayed"):
            state["replayed"] = True
            out.write("MITM: перехоплений запит клієнта надіслано серверу вдруге "
                      "атака( повторного відтворення)\n")
            out.flush()
            try:
                state["srv_sock"].sendall(state["captured"])
            except OSError:
                pass
```

```
            try:
                dst.sendall(data)
            except OSError:
                break
        dst.close()


def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("--listen", type=int, default=1600)
    ap.add_argument("--server", type=int, default=1502)
    ap.add_argument("--out", default="data/sniff.txt")
    ap.add_argument("--flip", action="store_true", help="підробити байт у першому запиті")
    ap.add_argument("--replay", action="store_true", help="повторно надіслати перехоплений
    ↪ запит")
    a = ap.parse_args()

    ls = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    ls.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    ls.bind(("127.0.0.1", a.listen))
    ls.listen(1)
    print(f"sniffer: 127.0.0.1:{a.listen} -> 127.0.0.1:{a.server}, лог у {a.out}")

    out = open(a.out, "w")
    client, _ = ls.accept()
    server = socket.create_connection(("127.0.0.1", a.server))
    mode = "flip" if a.flip else "replay" if a.replay else "passive"
    state: dict = {}
    t1 = threading.Thread(target=pump_attack if mode != "passive" else pump,
                          args=(client, server, "C->S", out),
                          kwargs={"mode": mode, "state": state} if mode != "passive" else {})
    t2 = threading.Thread(target=pump_attack if mode != "passive" else pump,
                          args=(server, client, "S->C", out),
                          kwargs={"mode": mode, "state": state} if mode != "passive" else {})
    t1.start(); t2.start()
    t1.join(); t2.join()
    out.close()
    print("sniffer: зєднання' закрито")


if __name__ == "__main__":
    main()
```

Додаток В. Відтворення сценаріїв (`run.sh`)

```
#!/usr/bin/env bash
# Відтворити всі сценарії й зібрати перехоплений трафік.
#   до захисту:   клієнт -> сніфер -> сервер відкритий( Modbus)
#   після захисту: те саме через TLS
#   атаки: підміна байта (--flip), повторне відтворення (--replay),
#          підробний сервер сертифікат( не від нашого CA)
#   продуктивність: --bench час( зєднання' та запитвідповідь-)
set -euo pipefail
cd "$(dirname "$0")"

gcc -O2 -Wall -Wextra -o modbus_tcp src/modbus_tcp.c -lssl -lcrypto
mkdir -p certs data
[ -f certs/server.crt ] || bash gen-certs.sh
mkdir -p data
PY=python3

run_case() {  # name tls [sniffer args...] [client args: --hold N / --repeat N]
  local name=$1 tls=$2; shift 2
  local sn_args=() cl_args=()
  while [ $# -gt 0 ]; do
    case "$1" in
      --hold|--repeat) cl_args+=("$1" "$2"); shift 2;;
      *) sn_args+=("$1"); shift;;
    esac
  done
  ./modbus_tcp --server $tls --port 1502 >/dev/null 2>&1 & SRV=$!
  sleep 0.5
  $PY src/sniffer.py --listen 1600 --server 1502 --out "data/sniff-$name.txt" \
      "${sn_args[@]}" & SNF=$!
  sleep 0.5
  ./modbus_tcp --client $tls --port 1600 "${cl_args[@]}" >"data/client-$name.txt" 2>&1 || true
  sleep 0.3
  kill $SNF $SRV 2>/dev/null || true
  wait $SNF 2>/dev/null || true
  echo "== $name: $(wc -l < data/sniff-$name.txt) рядків дампу"
}

# базове спостереження пассивний( сніфер)
run_case plain ""
run_case tls "--tls"

# атака 1: підміна команди байт( початковоі адреси у запиті клієнта)
run_case plain-flip "" --flip
run_case tls-flip "--tls" --flip

# атака 2: повторне відтворення перехопленого запиту клієнт( тримає зєднання')
run_case plain-replay "" --replay --hold 700
```

```
run_case tls-replay "--tls" --replay --hold 700

# атака 3: підробний сервер із сертифікатом не від нашого CA
if [ ! -f certs/evil.key ] || [ ! -f certs/evil.crt ]; then
  openssl ecparam -name prime256v1 -genkey -noout -out certs/evil.key
  openssl req -x509 -new -key certs/evil.key -sha256 -days 365 -out certs/evil.crt \
      -subj "/C=UA/O=Evil/CN=plc.local"
fi
./modbus_tcp --server --tls --port 1502 --cert certs/evil.crt --key certs/evil.key \
    >/dev/null 2>&1 & SRV=$!
sleep 0.5
./modbus_tcp --client --tls --port 1502 >data/client-evil-server.txt 2>&1 || true
kill $SRV 2>/dev/null || true
echo "== evil-server: клієнт відмовився: $(tail -1 data/client-evil-server.txt)"

# продуктивність: 200 зєднань', по 20 запитів
./modbus_tcp --server --port 1502 >/dev/null 2>&1 & S1=$!
./modbus_tcp --server --tls --port 1503 >/dev/null 2>&1 & S2=$!
sleep 0.5
{
  echo "== plain"
  ./modbus_tcp --bench 200 --port 1502
  echo "== TLS 1.3"
  ./modbus_tcp --bench 200 --tls --port 1503
} > data/bench.txt 2>&1
kill $S1 $S2 2>/dev/null || true
echo "готово: data/sniff-*.txt, data/client-*.txt, data/bench.txt"
```